

Prompt Engineering

Techniques from Basic to Advanced

Anthropic Docs | Wei et al. CoT | OpenAI | DeepMind | 2024

Introduction

Prompt engineering is the practice of designing inputs to language models to elicit desired outputs reliably. This document covers techniques from basic to advanced, drawing from Anthropic's official documentation (docs.anthropic.com) and published research.

1. Core Prompting Techniques

Zero-shot prompting: Direct instruction with no examples. Works for clear, common tasks.
'Classify the sentiment of this review: [review]'

Few-shot prompting: Provide 2–10 examples in the prompt (in-context learning). Dramatically improves accuracy for formatting, rare tasks, and domain-specific outputs.

Chain-of-thought (CoT): Ask model to 'think step by step' before answering. Improves math, logic, and multi-step reasoning. Wei et al. (2022) showed 5x improvement on GSM8K.

Zero-shot CoT: Simply append 'Let's think step by step.' Kojima et al. (2022) showed this works without examples.

Self-consistency: Sample multiple CoT chains, take majority vote. Wang et al. (2022).

Tree of Thoughts (ToT): Explore multiple reasoning paths, evaluate intermediate steps. Yao et al. (2023).

ReAct: Interleave reasoning (Thought:) and actions (Action:) with tool use. Yao et al. (2022). Foundation of most agent frameworks.

2. Anthropic Prompt Engineering Guidelines

From docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview:

Be direct and specific: State exactly what you want. Avoid vague instructions like 'help me'.

Use XML tags: Structure complex prompts with `<code>`, `</code>` tags. Claude is trained to recognize these and respond accordingly.

Give context: Explain the task's purpose, the intended audience, and success criteria.

Positive framing: 'Write clearly' is better than 'Don't write confusingly'.

Specify output format: Say 'respond in JSON with keys: name, score, reason' not 'format nicely'.

Separate instructions from content: Use tags or clear delimiters to distinguish instructions from user-supplied content (prevents prompt injection).

Use system prompts for personas: Set persistent context (role, constraints, format) in the system prompt; keep user turns focused on the task.

3. Prompt Template: Structured Reasoning

You are an expert data analyst. You respond concisely and cite evidence from the data.

Always structure your response as JSON with keys: analysis, confidence, recommendation.

Analyze the following sales data and identify the top growth opportunity.

Q1: \$1.2M | Q2: \$1.4M | Q3: \$1.1M | Q4: \$1.8M

Product A: 45% of revenue, declining

Product B: 30% of revenue, growing 20% QoQ

Product C: 25% of revenue, new launch

- Focus on actionable recommendations
- Consider both revenue and growth rate
- Limit analysis to 3 key points

4. Extended Thinking (Claude)

Claude 3.7 Sonnet introduced extended thinking: before responding, Claude can reason at length in a scratchpad (thinking tokens). This dramatically improves performance on hard math, coding, and multi-step reasoning tasks. Enable with:

```
thinking={"type": "enabled", "budget_tokens": 10000}
```

5. Prompt Injection & Security

Prompt injection occurs when user-supplied content contains instructions that override the system prompt. Mitigations: use XML tags to separate content, instruct model to ignore instructions in user-provided data, validate and sanitize outputs, use Anthropic's prompt injection detection guidelines.